

Universidad Tecnológica de Manzanillo

Ingeniería en Desarrollo y Gestión de Software

Materia: Desarrollo para dispositivos inteligentes

Alumnos:

Emmanuel Valencia Vázquez

Ulises Alejandro Curiel Pérez

Docente: Daniel German Fausto Solis

Grado: 9 Grupo: IDG 1

Manual técnico de MediaNest TV

Versión: 0.1.0

Fecha de elaboración: 18 de julio de 2026

Tabla de contenido

1. Introducción
2. Requisitos técnicos
3. Arquitectura del sistema
4. Estructura del código fuente
5. Instalación y ejecución
6. Funcionamiento interno y módulos
7. Datos, seguridad y configuración
8. Pruebas técnicas
9. Mantenimiento y solución de fallas
10. Versionado y anexos técnicos

1. Introducción

Este manual técnico documenta MediaNest TV, una aplicación web para pantallas LG con webOS. Su propósito es permitir que un desarrollador o responsable de soporte pueda instalar, configurar, ejecutar, diagnosticar y mantener el proyecto sin depender de explicaciones externas. La documentación describe la versión 0.1.0 disponible en el repositorio <https://github.com/Sage1985/medianest>.

1.1 Identificación del sistema

Campo	Valor
Nombre del sistema	MediaNest TV
Versión	0.1.0
Tipo de aplicación	Aplicación web nativa para LG webOS
Autores	Emmanuel Valencia Vázquez y Ulises Alejandro Curiel Pérez
Fecha de elaboración	18 de julio de 2026
Repositorio fuente	https://github.com/Sage1985/medianest

1.2 Descripción y alcance

MediaNest TV reúne películas abiertas, cortometrajes, series de prueba y música en una interfaz de 1280 × 720 diseñada para control remoto. El usuario puede cambiar de sección, buscar, abrir detalles, seleccionar episodios, reproducir medios y guardar favoritos o progreso local. La app se entrega como código web y puede empaquetarse como .ipk para instalarse en una TV LG en Developer Mode.

No es un sistema escolar de alumnos o calificaciones. Tampoco incluye login, roles, servidor propio, base de datos SQL/NoSQL ni panel administrativo. Esta delimitación es importante: las funciones de usuarios, contraseñas, importación de base de datos y respaldo de tablas no aplican a la versión actual.

2. Requisitos técnicos

2.1 Hardware y software

Elemento	Mínimo	Recomendado / observación
Equipo de desarrollo	CPU de 2 núcleos, 4 GB RAM, 1 GB libres	Windows 10/11 de 64 bits, 8 GB RAM y conexión a Internet.
Equipo objetivo	TV LG compatible con webOS	LG 32LM570BPUA o simulador webOS; control remoto funcional.
Sistema operativo	Windows 10/11, macOS o Linux	La guía entregada se probó desde Windows.
Node.js	Versión 18 o posterior	npm incluido; requerido por package.json.
Navegador de desarrollo	Chrome, Edge o Firefox moderno	Usar ventana de 1280 × 720 para revisar la interfaz.

Elemento	Mínimo	Recomendado / observación
Espacio de disco	500 MB	Incluye node_modules, herramientas de webOS y paquetes de prueba.

2.2 Dependencias y herramientas

Categoría	Tecnología	Uso técnico
Lenguajes	HTML5, CSS3 y JavaScript ES5	Estructura, estilos y lógica sin módulos modernos.
Plataforma	LG webOS y @webos-tools/cli	Empaquetado .ipk, instalación, lanzamiento e inspección.
Servidor de desarrollo	http-server 14	Sirve la carpeta app en http://localhost:8080; no se requiere Apache o Nginx.
API del navegador	fetch, localStorage, HTMLMediaElement	Consulta de datos, persistencia local y reproducción.
Servicios externos	Internet Archive, TVMaze, iTunes, Audius, Jamendo y YouTube	Catálogo, búsqueda o fuentes de audio/video; requieren red.
Base de datos	No aplica	No hay MySQL, PostgreSQL, Firebase ni scripts de importación.

3. Arquitectura del sistema

MediaNest utiliza una arquitectura cliente orientada a servicios. El cliente webOS concentra la interfaz y la lógica; consulta proveedores externos directamente por HTTPS y guarda únicamente estado no sensible en localStorage. No hay capa backend, servidor de aplicaciones ni API propia. Esta decisión reduce infraestructura para el prototipo, pero traslada la disponibilidad del catálogo a los proveedores externos.

Arquitectura lógica de MediaNest TV

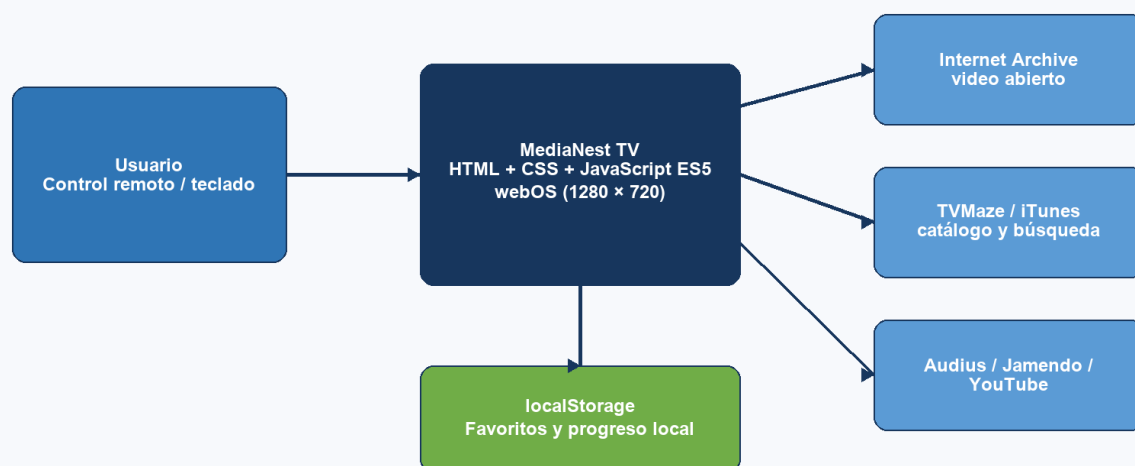


Figura T1. Arquitectura técnica de MediaNest TV.

3.1 Componentes y flujo de información

Componente	Responsabilidad	Entrada / salida
Interfaz de TV	Renderiza menú, tarjetas, diálogos y foco del control remoto.	Eventos de teclado/remoto → acciones de usuario.
Lógica app.js	Mantiene estado, consulta servicios, construye tarjetas y abre reproductor.	Acciones → cambios de DOM, consultas y persistencia.
Proveedores externos	Entregan resultados y medios abiertos o embebibles.	Solicitudes fetch → JSON, URL de imagen o medio.
Reproductor HTML5	Reproduce video/audio/iframe y registra el avance.	URL de medio → tiempo actual y estado.
localStorage	Conserva favoritos y recientes del dispositivo.	JSON local → interfaz Mi espacio.

El flujo normal inicia en index.html. Los scripts data.js y config.js cargan antes de app.js para que éste pueda usar el catálogo inicial y la configuración. Al elegir una sección o búsqueda, app.js solicita resultados, normaliza los datos y actualiza el DOM. Al reproducir, registra progreso en localStorage y el usuario puede volver con BACK.

4. Estructura del código fuente

Ruta	Función
/app	Raíz que se empaqueta para webOS; contiene la aplicación ejecutable.
/app/index.html	Documento principal, menú lateral, filas de catálogo y capas de detalle/reproductor.
/app/styles/main.css	Diseño visual de 1280 × 720, tarjetas, foco, menús y adaptaciones de interfaz.
/app/scripts/app.js	Lógica central: navegación, APIs, búsqueda, reproductor, favoritos y manejo de teclas.
/app/scripts/data.js	Catálogo inicial y contenido abierto usado como demostración y respaldo.
/app/scripts/config.js	Parámetros opcionales de integración; no se deben subir claves reales.
/app/assets	Iconos SVG pequeño y grande requeridos por la plataforma.
/package.json	Comandos npm, dependencias de desarrollo y versión del proyecto.
/GUIA_SIMULADOR_LG.md	Instrucciones complementarias para prueba en simulador LG.
/Documentos	Manual técnico, documento general y evidencias de entrega.

4.1 Archivos relevantes y responsabilidad

El estado global, los códigos de control remoto y las funciones de render se encuentran en app/scripts/app.js. Las claves LEFT, RIGHT, UP, DOWN, ENTER y BACK se convierten en

navegación de foco. `app/scripts/data.js` define objetos de contenido con campos `id`, `title`, `section`, `description`, `image`, `source` y `mediaType`. Antes de modificar esos archivos se recomienda preservar el formato JSON y revisar que las URLs usen HTTPS.

5. Instalación y ejecución

5.1 Preparar el entorno local

- Descargar el proyecto desde <https://github.com/Sage1985/medianest> o clonar el repositorio con Git.
- Abrir una terminal en la carpeta raíz del proyecto.
- Ejecutar `npm install` para instalar `http-server` y la CLI de webOS indicada en `package.json`.
- Ejecutar `npm run dev` y abrir `http://localhost:8080` en un navegador moderno.
- Probar con las flechas, Enter y Escape; ajustar el tamaño de navegador a 1280 × 720 si se requiere una simulación visual.

5.2 Configuración

No se configura servidor, dominio ni base de datos. El único archivo de configuración es `app/scripts/config.js`. `jamendoClientId` y `youtubeApiKey` permanecen vacíos en el repositorio público; si se requieren dichas integraciones, el responsable debe colocar una credencial propia solo en una copia local o en una capa de servidor. `youtubePlayerUrl` indica la URL usada por el reproductor embebido. Nunca se deben publicar claves API, contraseñas o archivos de firma en GitHub.

5.3 Empaquetado e instalación en webOS

- Instalar o configurar LG webOS CLI y registrar la TV con `ares-setup-device` cuando esté en la misma red.
- Ejecutar `npm run package:webos` para crear el archivo `.ipk` desde la carpeta `app`.
- Ejecutar `npm run install:webos` para enviar el paquete al dispositivo `school-lg-tv` configurado.
- Ejecutar `npm run launch:webos` para iniciar MediaNest en la TV.
- Usar `npm run inspect:webos` cuando sea necesario revisar la consola y la inspección remota.

6. Funcionamiento interno y módulos

6.1 Navegación e interfaz

El módulo de navegación administra `state.section`, `state.focusIndex` y el conjunto de elementos focusable. Cada tecla remota se traduce a una acción; el foco visible permite saber qué control se activará. Los archivos involucrados son `index.html` para los elementos interactivos, `main.css` para el estilo de foco y `app.js` para las funciones de evento y cambio de sección.

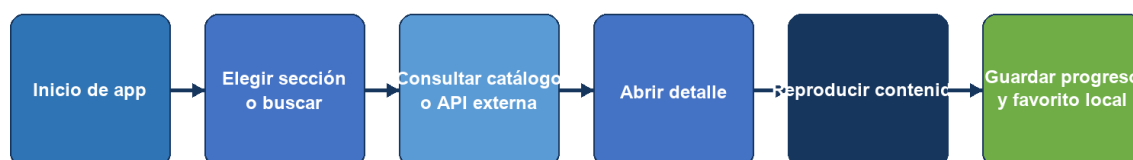
6.2 Catálogo, series y búsqueda

El catálogo inicial proviene de data.js. La búsqueda usa fetch y las rutas declaradas en API dentro de app.js para consultar Internet Archive, TVMaze, iTunes, Audius, Jamendo o YouTube según el tipo. Los resultados se normalizan para conservar una forma común de tarjeta. Si un proveedor falla, el sistema debe mostrar el estado correspondiente sin detener la navegación local.

6.3 Reproducción y Mi espacio

El reproductor abre playerOverlay y usa etiquetas video, audio o iframe. Las acciones permiten reproducir/pausar, retroceder 10 segundos, avanzar 30 segundos, guardar y salir. writeStore guarda las listas medianest.saved.v2 y medianest.recent.v2; cada registro contiene una copia del elemento y, cuando corresponde, el avance. No existe sincronización entre televisores.

Flujo de operación principal



BACK cierra reproductor o detalle; las flechas cambian el foco y OK/Enter confirma la acción.

Figura T2. Flujo técnico de navegación, consulta y reproducción.

7. Datos, seguridad y configuración

7.1 Modelo de datos local

Estructura	Campos importantes	Uso
Contenido JSON	id, title, kind, section, image, description, source, mediaType	Representa una película, serie, episodio o audio.
Favorito local	id y datos del contenido	Se guarda en medianest.saved.v2.
Reciente local	id, progreso y datos del contenido	Se guarda en medianest.recent.v2 y permite continuar.
Episodio	id, title, durationText, description, source	Se incluye dentro del arreglo episodes de una serie.

No hay modelo ER, tablas ni relaciones SQL porque no existe una base de datos. La relación lógica es contenido → episodios y contenido → registros locales de favorito/reciente. Para

respaldar el estado local se puede exportar localStorage desde las herramientas del navegador; para restaurarlo se vuelve a cargar el JSON válido bajo las mismas claves. Esta operación es manual y solo aplica al perfil del navegador o TV que creó los datos.

7.2 Seguridad y validación

- Autenticación, contraseñas y roles: no aplican; MediaNest no procesa cuentas de usuario.
- Credenciales: las claves externas se dejan vacías en el repositorio y no deben incrustarse en una publicación pública.
- Validación: requestJson verifica response.ok; safeText escapa texto mostrado y normalizeSearch limpia la consulta.
- Acceso a datos: localStorage sólo es accesible desde el origen de la aplicación; no contiene datos personales requeridos por el sistema.
- Contenido externo: usar HTTPS y comprobar las políticas de cada proveedor antes de distribuir una versión comercial.

8. Pruebas técnicas

Tipo de prueba	Caso	Resultado esperado
Sintaxis	node --check sobre app.js, data.js y config.js	Los archivos JavaScript no presentan errores de sintaxis.
Navegación	Flechas y Enter sobre menú, filtros y tarjetas	El foco se mueve y activa la acción correspondiente.
Reproductor	Pausa, avance, retroceso y BACK	El medio responde o el error se muestra sin bloquear la interfaz.
Persistencia	Guardar favorito, recargar y abrir Mi espacio	El registro continúa en el mismo origen local.
Integración	Buscar con conexión y sin conexión	Con red se muestran resultados; sin red se conserva navegación y catálogo inicial.
Empaquetado	npm run package:webos	Se genera un .ipk cuando la CLI está instalada y configurada.

9. Mantenimiento y solución de fallas

9.1 Actualización y respaldos

- Actualizar código: obtener los cambios del repositorio, ejecutar npm install si package-lock.json cambió y repetir las pruebas técnicas.
- Actualizar la TV: volver a generar el .ipk, instalarlo y lanzar la app; el paquete anterior puede sustituirse durante la instalación.
- Respaldo: conservar una copia del repositorio y del .ipk. No hay respaldo de base de datos; los favoritos se respaldan exportando localStorage de forma manual.

- Mantenimiento: revisar disponibilidad de APIs, vigencia de claves locales, compatibilidad de medios y versión de Node/webOS CLI.

9.2 Errores comunes

Problema	Causa probable	Solución
npm no reconocido	Node.js no está instalado o no está en PATH.	Instalar Node.js LTS y abrir una terminal nueva.
No carga catálogo en línea	Sin conexión, límite de API, CORS o proveedor no disponible.	Comprobar red, consola del navegador y probar otra búsqueda; el catálogo inicial seguirá disponible.
No reproduce un medio	URL vencida, codec no compatible o bloqueo del proveedor.	Validar URL HTTPS, probar una fuente abierta y revisar consola.
La TV no instala el .ipk	Dispositivo no registrado o Developer Mode vencido.	Ejecutar ares-setup-device, renovar Developer Mode y solicitar la llave.
No aparecen favoritos	Se borró el almacenamiento del navegador o cambió el origen.	Restaurar localStorage si existe respaldo; de lo contrario guardar nuevamente.

10. Versionado y anexos técnicos

Versión	Fecha	Cambios
0.1.0	Julio de 2026	Versión inicial: navegación TV, catálogo abierto, búsqueda, detalle, reproducción, favoritos, progreso local, configuración webOS y documentación técnica.
Próxima versión	Por definir	Posibles mejoras: backend para proteger claves, autenticación opcional, sincronización entre dispositivos, caché, control parental y pruebas automatizadas.

Anexos entregados: código fuente en <https://github.com/Sage1985/medianest>, appinfo.json, package.json, package-lock.json, guía de simulador LG, Documento General y Manual Técnico en Word/PDF. No se incluyen scripts de base de datos porque el sistema no utiliza una base de datos.